LAB REPORT

Linked List in Python

Addition, Insertion, Deletion, Searching, Updating and Basic Operations

Course	Data Structures and Algorithms Using Python
Lab No.	Lab 02
Student Name	Jiayu Yang （ID 20253801028）
Student ID	2025-AI-002
Submission Date	14 October 2026

1-1. Objective

The objective of this lab is to practice fundamental linked list operations in Python, including traversal, addition, insertion, deletion, searching, updating, aggregation, sorting and reversing. The lab also
demonstrates how these operations can be combined into a menu-driven Linked List Management System.

1-2. Software / Environment

Programming Language	Python 3.x
Editor / IDE	Editor
Data Structure	Python list
File Name	20253801028_Linked List.py

2-1. Task 1 - LAB MAIN MENU、Create and Display an Listed List

Requirement: Function Task1 Accept 10 integer values, store them in a list, and display the complete list, first element and last element.

Python code

```
def userInput_DispListedList():
    listedList = LinkedList()
    print("Jiayu Yang,ID:20253801028")
    print("¥n===== createLinkedList: Create and Display an ListedList =====")
    print("Input 10 elements:")
    numbers = list(map(int, input().split()))
    for n in numbers:
        listedList.append(n)
    print("List: ", end="")
    listedList.display()
    print(f"First: {listedList.get_first()}  Last: {listedList.get_last()}")
    return listedList
```

```
PS D:\Data-Structure-in-python\lab2> python .\20253801028_singly_linked_list_operations.py
List: 10 -> 20 -> 30 -> 40 -> 60 -> None

======== LAB MAIN MENU ========
 1: Create & Display linked list
 2: Add Node at End
 3: Add Node at Beginning
 4: Insert Node at Position
 5: Delete Node by Position
 6: Delete Node by Value
 7: Search Node
 8: Update Node
 9: Count Nodes
10: Maximum,Minimum,Sum,Average
11: Reverse List
12: linked list management system
13: Exit

Please select task number: 1
Jiayu Yang,ID:20253801028

===== createLinkedList: Create and Display an ListedList =====
Input 10 elements:
9 8 7 6 4 5 11 22 33 44
List: List: 9 -> 8 -> 7 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> None
First: 9  Last: 44
```

Figure 1. execution for Create & Display Linked List

Result: The program correctly stored ten integers and accessed the first and last positions using list indexing.

2-2. Task 2 -11 LinkedList class -Addition, Insertion,Deletion,Search and Update,Node Count,Sum_Value、Average_Value、Max、Min,Reverse List

Function Task2-3 demonstrates adding an element to the end/front of the list, Function Task4 inserting an element at a selected index, and Function Task5-6 deleting an element using its index and value. Function Task7 *searches for a user-provided value. If the value exists, its index is reported. Function Task8 selected element is then replaced by a new value. Task9 Count nodes number, count even/odd node value. Task 10 Max、Min、Sum、Average of the node value.Task11 reverse the linked list*

Python code

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
    def display(self):
        print("List: ", end="")
        current = self.head
        while current:
            print(current.data, end=" -> ")
            current = current.next
        print("None")
    def append(self, value):
        new_node = Node(value)
        if self.head is None:
            self.head = new_node
            return
        current = self.head
        while current.next:
            current = current.next
        current.next = new_node
        return True
    def prepend(self, value):
        new_node = Node(value)
        new_node.next = self.head
        self.head = new_node
        return True
    def insert_at(self, position, value):
        if position < 0:
            return False
        if position == 0:
            new_node = Node(value)
            new_node.next = self.head
            self.head = new_node
            return True
        previous = self.head
        for _ in range(position - 1):
            if previous is None:
                return False
            previous = previous.next
        if previous is None:
            return False
        new_node = Node(value)
        new_node.next = previous.next
        previous.next = new_node
        return  True
    def delete_value(self, value):
```

```
        if self.head is None:
            return False
        if self.head.data == value:
            self.head = self.head.next
            return True
        previous = self.head
        while previous.next:
            if previous.next.data == value:
                previous.next = previous.next.next
                return True
            previous = previous.next
        return False
    def delete_at(self, position):
        if position < 0 or self.head is None:
            return False
        if position == 0:
            self.head = self.head.next
            return True
        previous = self.head
        for _ in range(position - 1):
            if previous.next is None:
                return False
            previous = previous.next
        if previous.next is None:
            return False
        previous.next = previous.next.next
        return True
    def search(self, value):
        position = 0
        current = self.head
        while current is not None:
            if current.data == value:
                return position
            current = current.next
            position += 1
        return -1
    def update_at(self, position, new_value):
        if position < 0:
            return False
        current = self.head
        for _ in range(position):
            if current is None:
                return False
            current = current.next
        if current is None:
            return False
        current.data = new_value
        return True
    def count_nodes(self):
        count = 0
        current = self.head
        while current:
            count += 1
            current = current.next
        return count
    def count_even_nodes(self):
        count = 0
        current = self.head
        while current:
            if current.data%2 == 0:
                count += 1
            current = current.next
        return count
    def count_odd_nodes(self):
        count = 0
        current = self.head
        while current:
            if current.data%2:
                count += 1
            current = current.next
        return count
    def get_sum(self):
```

```
        total = 0
        current = self.head
        while current:
            total += current.data
            current = current.next
        return total
    def get_max(self):
        if self.head is None:
            return None
        max_val = self.head.data
        current = self.head.next
        while current:
            if current.data > max_val:
                max_val = current.data
            current = current.next
        return max_val
    def get_min(self):
        if self.head is None:
            return None
        min_val = self.head.data
        current = self.head.next
        while current:
            if current.data < min_val:
                min_val = current.data
            current = current.next
        return min_val
    def get_average(self):
        n = self.count_nodes()
        if n == 0:
            return None
        return self.get_sum() / n
    def reverse(self):
        previous = None
        current = self.head
        while current:
            next_node = current.next
            current.next = previous
            previous = current
            current = next_node
        self.head = previous
```

```
===== createLinkedList: Create and Display an ListedList =====
Input 10 elements:
9 8 7 6 4 5 11 22 33 44
List: List: 9 -> 8 -> 7 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> None
First: 9  Last: 44

Please select task number: 2
Enter element to add: 55
List: 9 -> 8 -> 7 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> 55 -> None

Please select task number: 3
Enter element to add: 66
List: 66 -> 9 -> 8 -> 7 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> 55 -> None

Please select task number: 4
Enter data to index: 1
Enter element to add: 77
List: 66 -> 77 -> 9 -> 8 -> 7 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> 55 -> None

Please select task number: 5
Enter index to delete: 2
List: 66 -> 77 -> 8 -> 7 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> 55 -> None

Please select task number: 6
Enter element to delete: 8
List: 66 -> 77 -> 7 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> 55 -> None

Please select task number: 7
List: 66 -> 77 -> 7 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> 55 -> None
Enter element to search: 5
search value success
```

Figure 2. Addition end、front, insertion and deletion, Search output

```
Please select task number: 8
List: 66 -> 77 -> 7 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> 55 -> None
Enter data to index: 2
Enter element to add: 88
update value success

Please select task number: 9
count_nodes: 11
count_odd_nodes: 5
count_even_nodes: 6

Please select task number: 10
get_max value: 88
get_min value: 4
total_value: 411
average_value: 37.36363636363637

Please select task number: 11
List: 66 -> 77 -> 88 -> 6 -> 4 -> 5 -> 11 -> 22 -> 33 -> 44 -> 55 -> None
List: 55 -> 44 -> 33 -> 22 -> 11 -> 5 -> 4 -> 6 -> 88 -> 77 -> 66 -> None
```

Figure 3. Update, Sum and Average, Count Even and odd node value, Reverse execution output

3. Test Cases

For the default Linked List[10, 20, 30, 40, 60]

1. FUNCTION Task1 Create & Display Linked List [91,82,73,64,55,46,37,28,19,00].
2. FUNCTION Task3 Insertion value 11, index 12, insert failed
3. FUNCTION Task3 Insertion value 11, index 8, output [91,82,73,64,55,46,37,28,11,19,00]
4. FUNCTION Task5 Search value 20, search value failed
5. FUNCTION Task5 Search value 37, search value success
6. FUNCTION Task6 Update Index3, value 33, update value success

```
PS D:\Data-Structure-in-python\lab2> python .\20253801028_singly_linked_list_operations.py
List: 10 -> 20 -> 30 -> 40 -> 60 -> None

======== LAB MAIN MENU ========
 1: Create & Display linked list
 2: Add Node at End
 3: Add Node at Beginning
 4: Insert Node at Position
 5: Delete Node by Position
 6: Delete Node by Value
 7: Search Node
 8: Update Node
 9: Count Nodes
10: Maximum,Minimum,Sum,Average
11: Reverse List
12: linked list management system
13: Exit

Please select task number: 1
Jiayu Yang,ID:20253801028

===== createLinkedList: Create and Display an ListedList =====
Input 10 elements:
91 82 73 64 55 46 37 28 19 00
List: List: 91 -> 82 -> 73 -> 64 -> 55 -> 46 -> 37 -> 28 -> 19 -> 0 -> None
First: 91  Last: 0
```

```
Please select task number: 12
List: 91 -> 82 -> 73 -> 64 -> 55 -> 46 -> 37 -> 28 -> 19 -> 0 -> None
singly linked list management system
 1: Display linked list
 2: Add Node at End
 3: Add Node at Beginning
 4: Insert Node at Position
 5: Delete Node by Position
 6: Delete Node by Value
 7: Search Node
 8: Update Node
 9: Count Nodes
10: Find Maximum
11: Find Minimum
12: Calculate Sum
13: Calculate Average
14: Reverse List
15: Exit

Please select task number: 4
Enter data to index: 12
Enter element to add: 11
insert failed
```

```
Please select task number: 4
Enter data to index: 8
Enter element to add: 11
List: 91 -> 82 -> 73 -> 64 -> 55 -> 46 -> 37 -> 28 -> 11 -> 19 -> 0 -> None
```

```
Please select task number: 7
List: 91 -> 82 -> 73 -> 64 -> 55 -> 46 -> 37 -> 28 -> 11 -> 19 -> 0 -> None
Enter element to search: 20
search value failed
```

```
Please select task number: 7
List: 91 -> 82 -> 73 -> 64 -> 55 -> 46 -> 37 -> 28 -> 11 -> 19 -> 0 -> None
Enter element to search: 37
search value success
```

```
Please select task number: 8
List: 91 -> 82 -> 73 -> 64 -> 55 -> 46 -> 37 -> 28 -> 11 -> 19 -> 0 -> None
Enter data to index: 3
Enter element to add: 33
update value success
```

Test	Operation	Input	Expected	Status

2	Insert	Value11 at Index 12	insert failed	Pass
3	Insert	Value11 at Index 8	[91,82,73,64,55,46,37,28,11,19,00]	Pass
4	Search	20	search value failed	Pass
5	Search	37	search value success	Pass
6	Update	Index 3, Value 33	update value success	Pass

4. Lab tasks

attachment with 20253801028_singly_linked_list_operations.py and Section. 2 Screenshots showing program execution

5.Main Linked List Management System

A menu-driven program was developed to integrate the main operations. The program continues running until the user selects Exit and validates indexes before insertion, deletion or updating.

Final integrated program

```
def main_list_management(listedlist):
    listedlist.display()
    print("singly linked list management system")
    while True:
        print(" 1: Display linked list")
        print(" 2: Add Node at End")
        print(" 3: Add Node at Beginning")
        print(" 4: Insert Node at Position")
        print(" 5: Delete Node by Position")
        print(" 6: Delete Node by Value")
        print(" 7: Search Node")
        print(" 8: Update Node")
        print(" 9: Count Nodes")
        print("10: Find Maximum")
        print("11: Find Minimum")
        print("12: Calculate Sum")
        print("13: Calculate Average")
        print("14: Reverse List")
        print("15: Exit")
        opt = input("¥nPlease select task number: ")

        if opt == "1":
            listedlist.display()
        elif opt == "2":
            val = int(input("Enter element to add: "))
            if listedlist.append(val):
                listedlist.display()
            else:
                print("append failed")
        elif opt == "3":
            val = int(input("Enter element to add: "))
            if listedlist.prepend(val):
                listedlist.display()
            else:
```

```
            print("preappend failed")
        elif opt == "4":
            index = int(input("Enter data to index: "))
            val = int(input("Enter element to add: "))
            if listedlist.insert_at(index,val):
                listedlist.display()
            else:
                print("insert failed")
        elif opt == "5":
            index = int(input("Enter index to delete: "))
            if listedlist.delete_at(index):
                listedlist.display()
            else:
                print("delete at failed")
        elif opt == "6":
            val = int(input("Enter element to delete: "))
            if listedlist.delete_value(val):
                listedlist.display()
            else:
                print("delete value failed")
        elif opt == "7":
            listedlist.display()
            val = int(input("Enter element to search: "))
            if listedlist.search(val) != -1:
               print("search value success")
            else:
               print("search value failed")
        elif opt == "8":
            listedlist.display()
            index = int(input("Enter data to index: "))
            val = int(input("Enter element to add: "))
            if listedlist.update_at(index,val):
                print("update value success")
            else:
                print("update value failed")
        elif opt == "9":
            print("count_nodes:", listedlist.count_nodes())
            print("count_odd_nodes:", listedlist.count_odd_nodes())
            print("count_even_nodes:", listedlist.count_even_nodes())
        elif opt == "10":
            print("get_max value:", listedlist.get_max())
        elif opt == "11":
            print("get_min value:", listedlist.get_min())
        elif opt == "12":
            print("total_value:", listedlist.get_sum())
        elif opt == "13":
            print("average_value:", listedlist.get_average())
        elif opt == "14":
            listedlist.display()
            listedlist.reverse()
            listedlist.display()
        elif opt == "15":
            print("linked list management system")
            break
        else:
            print("Invalid option, try again.")
```

Figure 4. Example execution of the Main Linked List Management System

6. Answers to the questions

Q1. What is a singly linked list?

A singly linked list is a linear data structure made of nodes. Each node stores a value and a reference (next) to the following node. The last node points to None. The head pointer refers to the first node.

Q2. What are the two fields stored in a node?

A node stores two fields: data (the value) and next (a reference to the next node in the sequence).

Q3. What does head represent?

Head is a reference to the first node in the linked list. It serves as the entry point for all traversal, insertion, and deletion operations.

Q4. What does None mean in a linked list?

None in the next field of a node indicates that it is the last node. An empty list is represented by head being None.

Q5. How is traversal different from array indexing?

Array indexing allows direct access to any element in constant time. Linked list traversal must start at head and follow next references sequentially, resulting in O(n) access time for arbitrary positions.

Q6. Why do we need to handle insertion at position 0 separately?

Inserting at position 0 changes the head of the list. The new node must point to the old head, and the head reference must be updated. For other positions, only the next pointers of surrounding nodes need adjustment.

Q7. What is the difference between inserting at the beginning and at the end?

Inserting at the beginning (prepend) only requires creating a new node and updating head, which is O(1). Inserting at the end (append) requires traversing the entire list to find the last node, which is O(n) unless a tail pointer is maintained.

Q8. What happens when the value to delete is in the first node?

When the head node contains the target value, the head is updated to head next node, effectively removing the original first node from the list.

Q9. How do you delete a node without shifting all other nodes?

The previous node's next pointer is set to the deleted node's next, bypassing the deleted node. No shifting of other nodes is needed; only one pointer is updated.

Q10. What does searching return when a value is not found?

The search function returns -1 when the value is not found in any node and returns the zero-based position when found.

Q11. How can an existing node value be updated?

Traverse to the node at the specified position, then change its data field to the new value. The next pointer remains unchanged.

Q12. Why must the next pointer be preserved while reversing?

During reversal, the next node variable saves current next node before it is overwritten. Without this, the reference to the rest of the list would be lost, and the reversal would fail.

Q13. What is the time complexity of search and traversal?

Both search and traversal are O(n), where n is the number of nodes, because each must visit nodes sequentially from head to the target or to the end.

Q14. Compare array and singly linked list insertion, deletion, and random access.

Array insertion/deletion at arbitrary positions requires shifting elements (O(n)), while linked list insertion/deletion only requires pointer updates (O(1) if the position is known). However, random access in arrays is O(1) by index, while linked list access is O(n) by traversal.

Q15. What should the program do when the list is empty?

The program should check if head is None before performing operations. It should print a message such as "List is empty" and not crash. For append, it should be set head to the new node. For display, it should print "None". For search or statistics, it should return an appropriate message.